

SensiSpace

Design Document

Satellite Land Cover Classification System
Baby Dragon Hatchling vs ResNet-18 on EuroSAT

ISRO RESPOND Initiative - Major Project

Interface Design | Design Decisions | Non-Functional Aspects | SRS Traceability

1. Interface or Interaction Design

1.1 Frontend UI Design

The SensiSpace web application uses a premium dark-mode single-page interface built with HTML5, CSS3 (glassmorphism design), and vanilla JavaScript. The interface is designed for intuitive satellite image analysis with real-time feedback.

Layout Structure

Section	Purpose
Header	Sticky navbar with SensiSpace logo + ISRO RESPOND badge
Hero	Title + subtitle explaining the project at a glance
Upload Section	Drag-and-drop zone + file picker + sample loader
Results (3 tabs)	ResNet-18 / Baby Dragon Hatchling / Comparison
Footer	Project attribution and tech stack

User Interaction Flow

User lands on page

- > Uploads satellite image (drag-drop or file picker)
OR clicks "Load Random EuroSAT Sample"
- > Clicks "Run Classification"
- > System shows loading spinner
- > Results appear in 3 tabs:
 - Tab 1: ResNet-18 prediction + confidence + overlay + chart
 - Tab 2: BDH prediction + confidence + overlay + chart
 - Tab 3: Side-by-side comparison + benchmark table + legend

UI Design Elements

- **Glassmorphism:** Semi-transparent cards with backdrop-filter blur(24px) for depth
- **Animated Background:** 3 colored gradient orbs (purple, blue, green) for visual richness
- **Color-Coded Results:** Each land cover class has unique bright RGB color across strips, bars, overlays
- **Micro-Animations:** Fade-up on load, hover effects, animated probability bars, loading spinner
- **Responsive Design:** CSS media queries at 640px for mobile adaptation
- **Typography:** Inter (body) + Outfit (headings) from Google Fonts

1.2 API Design (RESTful)

The backend is built with FastAPI (Python ASGI framework), serving both the ML models and the static frontend files. CORS middleware is enabled for cross-origin requests.

Endpoint	Method	Output	Purpose
/api/predict	POST	JSON: both models results + overlay	Core classification
/api/sample	GET	JPEG image file	Random EuroSAT sample
/api/metrics	GET	JSON: accuracy, F1, params etc.	Training benchmarks
/api/colors	GET	JSON: {class: [R,G,B]}	Color palette
/api/health	GET	JSON: {status, loaded}	Health check

API Response Structure (/api/predict)

```
{
  "resnet": {
    "prediction": "Forest",
    "confidence": 0.8734,
    "inference_ms": 0.18,
    "all_probs": {"Forest": 0.87, "River": 0.05, ...},
    "color": [34, 220, 77],
    "description": "Dense tree cover...",
    "cam_image": "data:image/jpeg;base64,..."
  },
  "baby_dragon": { /* same structure */ }
}
```

1.3 Integrations

Integration	Purpose
PyTorch	Model loading, inference, tensor operations
torchvision	EuroSAT dataset, transforms, ResNet-18 weights
OpenCV	HSV/LAB conversion, morphology, contour detection
Pillow (PIL)	Image I/O, overlay rendering, label drawing
scikit-learn	Accuracy, precision, recall, F1 computation
Uvicorn	ASGI server for FastAPI deployment

2. Key Design Decisions, Assumptions, and Constraints

2.1 Design Decisions

Decision	Rationale
Custom Standard CNN	Standard CNNs are not purpose-built for satellite imagery; BDH modules target multi-scale objects and spectral boundaries
Spectral segmentation	Grayscale maps are blurry 8x8 blobs; HSV/LAB spectral analysis provides pixel-accurate boundaries
ResNet-18 frozen base	Represents common industry transfer learning approach for fair, realistic baseline comparison
64x64 input resolution	Matches EuroSAT native resolution; avoids upscaling artifacts; keeps inference fast
FastAPI over Flask/Django	Async support, auto OpenAPI docs, type validation, better ML model serving performance
Vanilla JS (no React/Vue)	Simple SPA with limited interactivity; framework adds unnecessary complexity and bundle size
Adam + CosineAnnealingLR	Adaptive LR for stable convergence; cosine annealing for better final accuracy
Dual dropout (0.3 + 0.2)	Prevents overfitting on 27K images without excessive regularization

2.2 Assumptions

Assumption	Justification
Input images are RGB satellite	EuroSAT provides RGB; spectral segmentation relies on color analysis
64x64 resolution sufficient	EuroSAT benchmark standard; 10m/pixel Sentinel-2 captures land cover
10 classes cover primary uses	EuroSAT classes represent major land types for ISRO monitoring
Single-label classification	Each EuroSAT image shows one dominant land type; multi-label is out of scope
CPU deployment sufficient	Ensures portability; inference is <1ms per image regardless

2.3 Constraints

Constraint	Impact and Mitigation
Dataset: 27,000 images	Sufficient for proof-of-concept; data augmentation mitigates limited size
RGB only (3 channels)	Sentinel-2 has 13 bands; future: extend to full multi-spectral
Image-level classification	Not pixel-level; spectral overlay provides approximate pixel mapping
No real-time satellite feed	Processes uploaded images only; future: integrate live data streams
CPU inference only	Limits batch throughput; sufficient for interactive single-image use

3. Non-Functional Aspects

3.1 Performance

Metric	ResNet-18	BDH
Inference latency	0.18 ms/image	0.60 ms/image
Throughput (theoretical)	~5,500 img/sec	~1,666 img/sec
Model load time	< 2 sec	< 1 sec
Frontend load time	< 1 sec	< 1 sec

- Both models achieve sub-millisecond inference on CPU -- real-time interactive performance
- FastAPI async architecture prevents blocking during image processing
- Frontend uses lazy rendering -- results section hidden until classification completes
- Probability bar animations use CSS transitions (GPU-accelerated), not JavaScript

3.2 Scalability

Aspect	Current	Scalability Path
Concurrent users	Single Uvicorn worker	Add --workers N or Gunicorn
Model serving	In-process PyTorch	TorchServe / Triton Inference Server
Dataset	27K (EuroSAT)	Swap for BigEarthNet (590K images)
Classes	10 (configurable)	Change num_classes parameter
Image size	64x64	Fully convolutional; works with any size
Horizontal scaling	Single server	Containerize (Docker) + load balance

3.3 Security

Concern	Mitigation
CORS policy	Currently allow_origins=["*"] for dev; restrict in production
File upload validation	FastAPI UploadFile + PIL format validation; no raw file writes
Authentication	None (academic demo); production would add API keys or OAuth
Injection attacks	No SQL/database; all data in-memory or JSON files -- no injection surface
Model weights	Stored as .pth files loaded at startup; not exposed via API
Input sanitization	Fixed transform pipeline (resize, normalize) prevents adversarial inputs

3.4 Reliability

- Graceful error handling: /api/predict returns HTTP 503 if models not loaded, with descriptive error
- Fallback fonts: Spectral overlay tries Helvetica -> DejaVuSans -> default font
- Auto device detection: Automatically selects CUDA -> MPS -> CPU based on availability
- Startup verification: Logs confirm each model loaded successfully with device info

4. Design with Traceability from SRS

4.1 Functional Requirements Traceability

Req ID	Requirement	Implementation
FR-1	Classify satellite images into land cov	baby_dragon.py, resnet.py
FR-2	Support 10 EuroSAT classes	CLASS_LABELS in train.py, num_classes=10
FR-3	Accept user-uploaded images	#fileInput in index.html, /api/predict POST
FR-4	Provide sample images for demo	/api/sample GET in app.py, #sampleBtn
FR-5	Compare two model architectures	Dual run_model() calls in /api/predict
FR-6	Visualize classification results	gradcam.py (spectral), app.js (charts)
FR-7	Display confidence scores	softmax() in app.py, stat-pill in HTML
FR-8	Show training benchmarks	metrics.json, renderMetrics() in app.js

4.2 Non-Functional Requirements Traceability

Req ID	Requirement	Implementation
NFR-1	Real-time inference (<1 sec)	BDH: 0.60ms, ResNet: 0.18ms per image
NFR-2	Responsive UI across devices	@media (max-width: 640px) in style.css
NFR-3	Handle missing models gracefully	HTTP 503 check in /api/predict
NFR-4	No external service dependency	All inference local; no cloud API calls
NFR-5	Fast page load	<1s load; 3 static files, no JS framework

4.3 Design Consistency Principles

Principle	How Enforced
Separation of Concerns	Models (src/models/), API (app.py), Frontend (static/), Training (train.py) fully separated
Single Responsibility	Each file has one job: architecture, baseline, visualization, or routing
Consistent API responses	Both models return identical JSON structure for frontend compatibility
Fair training pipeline	Both models use same optimizer, LR, loss, augmentation, eval metrics
Consistent color scheme	LAND_COVER_COLORS shared across backend overlays and frontend charts
Modular architecture	BDH stages are composable: MultiScale + LateralInhibition + Attention